

DECORATOR PATTERN

Notifier.java:

```
public interface Notifier {  
    void send();  
}
```

NotifierDecorator.java:

```
public abstract class NotifierDecorator implements Notifier {  
    protected Notifier notifier;  
    public NotifierDecorator(Notifier notifier) {  
        this.notifier=notifier;  
    }  
    @Override  
    public void send() {  
        notifier.send();  
    }  
}
```

SlackNotifierDecorator.java:

```
public class SlackNotifierDecorator extends NotifierDecorator {  
    public SlackNotifierDecorator(Notifier notifier) {  
        super(notifier);  
    }  
    @Override  
    public void send() {  
        super.send();  
        System.out.println("Sending Slack Notification!");  
    }  
}
```

SMSNotifierDecorator.java:

```
public class SMSNotifierDecorator extends NotifierDecorator {  
    public SMSNotifierDecorator(Notifier notifier) {  
        super(notifier);  
    }  
}
```

```

    }
    @Override
    public void send() {
        super.send();
        System.out.println("Sending SMS Notification!");
    }
}

```

EmailNotifier.java:

```

public class EmailNotifier implements Notifier {
    @Override
    public void send() {
        System.out.println("Sending Email Notification!");
    }
}

```

DecoratorPatternTest.java:

```

public class DecoratorPatternTest {
    public static void main(String[] args) {
        Notifier notify=
            new SlackNotifierDecorator(
                new SMSNotifierDecorator(
                    new EmailNotifier()
                )
            );
        notify.send();
    }
}

```

